

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	K.Prathiba	Reg. No.:	192421163
Date:	21-08-2026	Duration:	60 minutes

Code of Conduct

I K.Prathiba (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Prathiba

Annexure A – Test Case Design Assignment

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

1.FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS TO BE TESTED

1.1 Functional Requirements

Functional requirements describe the functions that the **AI-Based Personal Finance and Budget Planning System** must perform.

Requirement ID	Functional Requirement
FR01	The system shall allow users to register and create an account.
FR02	The system shall allow registered users to log in securely.
FR03	The system shall allow users to add income details.
FR04	The system shall allow users to add expense details.
FR05	The system shall allow users to categorize expenses.
FR06	The system shall allow users to create a monthly budget.
FR07	The system shall allow users to modify or update a budget.
FR08	The system shall calculate total income.
FR09	The system shall calculate total expenses.
FR10	The system shall calculate the remaining balance.
FR11	The system shall display transaction history.
FR12	The system shall allow users to edit transactions.
FR13	The system shall allow users to delete transactions.
FR14	The system shall display financial information through a dashboard.
FR15	The system shall analyse the user's spending pattern.
FR16	The system shall provide AI-based budget recommendations.
FR17	The system shall identify excessive or unusual spending.
FR18	The system shall provide budget alerts when spending approaches or exceeds the limit.
FR19	The system shall validate user-entered financial data.
FR20	The system shall handle invalid requests and display appropriate error messages.

1.2 Non-Functional Requirements

Requirement ID Non-Functional Requirement

NFR01	The system should be easy and simple for users to operate.
NFR02	The system should provide responses within a reasonable time.
NFR03	Financial calculations should be accurate.
NFR04	User financial information should be securely stored.
NFR05	The system should prevent unauthorized access.
NFR06	The system should protect user privacy.
NFR07	The system should be reliable during normal usage.
NFR08	The system should handle errors without crashing.
NFR09	The system should maintain data integrity.
NFR10	The application should work correctly on supported browsers and devices.
NFR11	The system should be scalable when the number of transactions increases.

Requirement ID Non-Functional Requirement

NFR12 The system should be maintainable and modular.

1.3 Requirements to be Tested

The main requirements selected for testing are:

- User registration and login.
- Income entry and validation.
- Expense entry and validation.
- Expense categorization.
- Budget creation and modification.
- Financial calculations.
- Transaction management.
- Dashboard display.
- Spending analysis.
- AI-based recommendations.
- Budget alerts.
- Security and privacy.
- Error handling.
- Data accuracy and integrity.
- System reliability and performance.

These requirements provide the foundation for preparing the test scenarios and detailed test cases.

2. TEST SCENARIOS FOR THE ASSIGNED PROBLEM STATEMENT

A test scenario represents a functionality or condition that needs to be verified.

Scenario ID Test Scenario

TS01	Verify user registration with valid details.
TS02	Verify registration with invalid or missing details.
TS03	Verify login with valid credentials.
TS04	Verify login with invalid credentials.
TS05	Verify adding valid income.
TS06	Verify invalid income values.
TS07	Verify adding valid expenses.
TS08	Verify invalid expense values.
TS09	Verify expense categorization.
TS10	Verify creation of monthly budget.
TS11	Verify invalid and boundary budget values.
TS12	Verify budget modification.
TS13	Verify total income calculation.

Scenario ID Test Scenario

TS14	Verify total expense calculation.
TS15	Verify remaining balance calculation.
TS16	Verify budget utilization calculation.
TS17	Verify transaction history.
TS18	Verify updating a transaction.
TS19	Verify deleting a transaction.
TS20	Verify dashboard financial summary.
TS21	Verify spending pattern analysis.
TS22	Verify AI-based financial recommendation.
TS23	Verify AI recommendation with insufficient data.
TS24	Verify budget alert when spending exceeds the limit.
TS25	Verify unauthorized access protection.
TS26	Verify error handling.
TS27	Verify data persistence and reliability.
TS28	Verify system performance.

3. DETAILED TEST CASES

The test cases cover **normal, boundary, invalid, and exceptional conditions**.

Each test case contains:

- Test Case ID
- Test Scenario
- Test Steps
- Test Data
- Expected Result
- Actual Result
- Pass/Fail Status

3.1 USER REGISTRATION AND LOGIN

TC01 – Valid User Registration

Test Case ID: TC01

Test Scenario: Verify registration using valid details.

Test Steps:

1. Open the application.
2. Select the Registration option.
3. Enter a valid name.
4. Enter a valid email address.
5. Enter a valid password.

6. Click Register.

Test Data:

Name: Prathiba

Email: prathiba@example.com

Password: ValidPassword123

Expected Result:

The system should validate the details and successfully create the user account.

Actual Result:

User account is created successfully.

Pass/Fail: PASS

TC02 – Empty Registration Fields

Test Case ID: TC02

Test Scenario: Verify registration with missing mandatory fields.

Test Steps:

1. Open the registration page.
2. Leave required fields empty.
3. Click Register.

Test Data: Blank name, email and password.

Expected Result:

The system should display validation messages and should not create the account.

Actual Result:

Validation messages are displayed.

Pass/Fail: PASS

TC03 – Invalid Email

Test Case ID: TC03

Test Scenario: Verify invalid email validation.

Test Steps:

1. Open registration.
2. Enter a valid name.
3. Enter an invalid email.
4. Enter a valid password.

5. Click Register.

Test Data: prathiba@

Expected Result:

The system should reject the invalid email format.

Actual Result:

Invalid email is rejected.

Pass/Fail: PASS

TC04 – Valid Login

Test Case ID: TC04

Test Scenario: Verify login with valid credentials.

Test Steps:

1. Open the login page.
2. Enter registered email.
3. Enter correct password.
4. Click Login.

Test Data: Valid registered credentials.

Expected Result:

The user should successfully log in and reach the finance dashboard.

Actual Result:

User successfully reaches the dashboard.

Pass/Fail: PASS

TC05 – Invalid Login

Test Case ID: TC05

Test Scenario: Verify login with incorrect password.

Test Steps:

1. Open login.
2. Enter registered email.
3. Enter incorrect password.
4. Click Login.

Test Data: Correct email + incorrect password.

Expected Result:

The system should reject the login and display an error message.

Actual Result:

Login is rejected and error message is displayed.

Pass/Fail: PASS

3.2 INCOME MANAGEMENT

TC06 – Add Valid Income

Test Case ID: TC06

Test Scenario: Verify adding income.

Test Steps:

1. Login to the system.
2. Open Income section.
3. Enter income source.
4. Enter amount.
5. Select date.
6. Save the income.

Test Data:

Source: Salary

Amount: ₹50,000

Expected Result:

Income should be stored successfully and total income should be updated.

Actual Result:

Income is stored and total income is updated.

Pass/Fail: PASS

TC07 – Zero Income

Test Case ID: TC07

Test Scenario: Verify boundary condition for income.

Test Steps:

1. Open Income.
2. Enter ₹0 as income.
3. Click Save.

Test Data: ₹0

Expected Result:

The system should reject zero if only positive income values are permitted.

Actual Result:

Zero income is rejected.

Pass/Fail: PASS

TC08 – Negative Income

Test Case ID: TC08

Test Scenario: Verify negative income validation.

Test Steps:

1. Open Income.
2. Enter a negative amount.
3. Click Save.

Test Data: -₹5,000

Expected Result:

The system should reject the negative amount.

Actual Result:

Negative amount is rejected.

Pass/Fail: PASS

TC09 – Non-Numeric Income

Test Case ID: TC09

Test Scenario: Verify invalid income format.

Test Steps:

1. Open Income.
2. Enter alphabetic characters in the amount field.
3. Click Save.

Test Data: ABCDE

Expected Result:

The system should reject non-numeric input.

Actual Result:

Invalid input is rejected.

Pass/Fail: PASS

3.3 EXPENSE MANAGEMENT

TC10 – Add Valid Expense

Test Case ID: TC10

Test Scenario: Verify adding an expense.

Test Steps:

1. Open Expense section.
2. Select an expense category.
3. Enter expense amount.
4. Select date.
5. Save.

Test Data:

Category: Food

Amount: ₹2,000

Expected Result:

Expense should be stored successfully.

Actual Result:

Expense is stored successfully.

Pass/Fail: PASS

TC11 – Empty Expense Amount

Test Case ID: TC11

Test Scenario: Verify empty expense amount.

Test Steps:

1. Open Expense.
2. Select category.
3. Leave amount blank.
4. Click Save.

Test Data: Blank amount.

Expected Result:

The system should display a required-field validation message.

Actual Result:

Validation message is displayed.

Pass/Fail: PASS

TC12 – Negative Expense

Test Case ID: TC12

Test Scenario: Verify negative expense validation.

Test Data: -₹1,000

Expected Result:

The system should reject negative expense values.

Actual Result:

Negative expense is rejected.

Pass/Fail: PASS

TC13 – Expense Categorization

Test Case ID: TC13

Test Scenario: Verify expense category selection.

Test Steps:

1. Open Expense.
2. Select Food category.
3. Enter amount.
4. Save transaction.
5. View transaction history.

Test Data: Food – ₹1,500

Expected Result:

The transaction should be stored under the Food category.

Actual Result:

Transaction is correctly categorized.

Pass/Fail: PASS

3.4 BUDGET MANAGEMENT

TC14 – Create Monthly Budget

Test Case ID: TC14

Test Scenario: Verify budget creation.

Test Steps:

1. Open Budget section.
2. Select month.
3. Select category.
4. Enter budget amount.
5. Click Save.

Test Data:

Category: Food

Monthly Budget: ₹10,000

Expected Result:

The system should create and display the monthly budget.

Actual Result:

Budget is created successfully.

Pass/Fail: PASS

TC15 – Zero Budget

Test Case ID: TC15

Test Scenario: Verify boundary value for budget.

Test Data: ₹0

Expected Result:

The system should reject zero if a positive budget is required.

Actual Result:

Zero budget is rejected.

Pass/Fail: PASS

TC16 – Negative Budget

Test Case ID: TC16

Test Scenario: Verify invalid budget amount.

Test Data: -₹5,000

Expected Result:

The system should reject negative budget values.

Actual Result:

Negative budget is rejected.

Pass/Fail: PASS

TC17 – Budget Exceeded

Test Case ID: TC17

Test Scenario: Verify overspending alert.

Test Steps:

1. Create a ₹5,000 budget.
2. Add expenses totalling ₹6,000.
3. View the budget dashboard.

Test Data:

Budget = ₹5,000

Expense = ₹6,000

Expected Result:

The system should identify that the budget has been exceeded and display an appropriate alert.

Actual Result:

Overspending alert is displayed.

Pass/Fail: PASS

3.5 FINANCIAL CALCULATIONS

TC18 – Total Income Calculation

Test Case ID: TC18

Test Scenario: Verify total income calculation.

Test Data:

Salary = ₹40,000

Freelance = ₹10,000

Expected Result:

Total Income = ₹50,000.

Actual Result:

Total Income displayed as ₹50,000.

Pass/Fail: PASS

TC19 – Total Expense Calculation

Test Case ID: TC19

Test Scenario: Verify total expense calculation.

Test Data:

Food = ₹5,000

Transport = ₹3,000

Shopping = ₹2,000

Expected Result:

Total Expense = ₹10,000.

Actual Result:

Total Expense displayed as ₹10,000.

Pass/Fail: PASS

TC20 – Remaining Balance

Test Case ID: TC20

Test Scenario: Verify remaining balance.

Test Data:

Income = ₹50,000

Expense = ₹15,000

Expected Result:

Remaining Balance = ₹35,000.

Actual Result:

Remaining Balance displayed as ₹35,000.

Pass/Fail: PASS

TC21 – Budget Utilization

Test Case ID: TC21

Test Scenario: Verify budget utilization calculation.

Test Data:

Budget = ₹20,000

Expense = ₹5,000

Expected Result:

Budget utilization should be 25%.

Actual Result:

25% utilization displayed.

Pass/Fail: PASS

3.6 TRANSACTION MANAGEMENT

TC22 – View Transaction History

Test Case ID: TC22

Test Scenario: Verify transaction history.

Test Steps:

1. Login.
2. Open Transaction History.
3. View saved records.

Test Data: Existing income and expense transactions.

Expected Result:

All valid transactions should be displayed correctly.

Actual Result:

Transactions are displayed correctly.

Pass/Fail: PASS

TC23 – Update Transaction

Test Case ID: TC23

Test Scenario: Verify transaction update.

Test Steps:

1. Select an existing transaction.
2. Click Edit.
3. Change the amount.
4. Save changes.

Test Data: Expense changed from ₹1,500 to ₹2,000.

Expected Result:

The updated transaction should be saved and totals should be recalculated.

Actual Result:

Transaction updated successfully.

Pass/Fail: PASS

TC24 – Delete Transaction

Test Case ID: TC24

Test Scenario: Verify transaction deletion.

Test Steps:

1. Select a transaction.
2. Click Delete.
3. Confirm deletion.

Test Data: Existing expense transaction.

Expected Result:

Transaction should be deleted and financial totals should be updated.

Actual Result:

Transaction deleted successfully.

Pass/Fail: PASS

3.7 AI-BASED FINANCIAL ANALYSIS

TC25 – AI Budget Recommendation

Test Case ID: TC25

Test Scenario: Verify AI-based budget recommendation.

Test Steps:

1. Add income information.
2. Add multiple expense records.
3. Create a budget.
4. Open AI recommendation feature.
5. Request recommendation.

Test Data:

Monthly Income = ₹50,000

Food = ₹8,000

Transport = ₹4,000

Entertainment = ₹6,000

Savings = ₹5,000

Expected Result:

The AI should analyse the available financial data and provide a relevant budgeting recommendation.

Actual Result:

AI recommendation is generated.

Pass/Fail: PASS

TC26 – Insufficient Financial Data

Test Case ID: TC26

Test Scenario: Verify AI behaviour with insufficient data.

Test Steps:

1. Login to the system.
2. Provide little or no transaction information.
3. Open AI recommendation.

Test Data: Insufficient transaction history.

Expected Result:

The system should inform the user that additional financial data is required instead of generating an unreliable recommendation.

Actual Result:

Insufficient-data message is displayed.

Pass/Fail: PASS

3.8 DASHBOARD AND SPENDING ANALYSIS

TC27 – Dashboard Summary

Test Case ID: TC27

Test Scenario: Verify financial dashboard.

Test Data:

Income = ₹50,000

Expense = ₹20,000

Balance = ₹30,000

Expected Result:

The dashboard should display the correct income, expense and balance.

Actual Result:

Correct financial summary is displayed.

Pass/Fail: PASS

TC28 – Spending Category Analysis

Test Case ID: TC28

Test Scenario: Verify category-wise spending analysis.

Test Data:

Food = ₹5,000

Transport = ₹3,000

Shopping = ₹2,000

Expected Result:

The system should correctly identify and display spending by category.

Actual Result:

Category-wise spending is displayed correctly.

Pass/Fail: PASS

3.9 SECURITY AND EXCEPTIONAL CONDITIONS

TC29 – Unauthorized Access

Test Case ID: TC29

Test Scenario: Verify protection of financial information.

Test Steps:

1. Login as User A.
2. Attempt to access another user's financial information.
3. Check system response.

Test Data: Unauthorized user/session.

Expected Result:

The system should deny unauthorized access.

Actual Result:

Unauthorized access is denied.

Pass/Fail: PASS

TC30 – Access After Logout

Test Case ID: TC30

Test Scenario: Verify session security.

Test Steps:

1. Login.
2. Open dashboard.
3. Logout.
4. Try to access dashboard again.

Test Data: Logged-out session.

Expected Result:

The system should redirect the user to the login page.

Actual Result:

Dashboard access is denied after logout.

Pass/Fail: PASS

TC31 – Backend Service Failure

Test Case ID: TC31

Test Scenario: Verify exceptional condition when backend is unavailable.

Test Steps:

1. Make the backend service unavailable.
2. Open the dashboard.
3. Request financial information.

Test Data: Backend unavailable.

Expected Result:

The system should display an appropriate error message and should not crash.

Actual Result:

Service error message is displayed.

Pass/Fail: PASS

4. TEST DESIGN TECHNIQUES

4.1 Equivalence Partitioning

Equivalence Partitioning divides input data into valid and invalid groups.

For financial amounts:

Partition	Example	Expected Result
Positive value	₹1,000	Valid
Zero	₹0	Invalid
Negative value	-₹500	Invalid
Alphabetic value	ABC	Invalid
Empty value	Blank	Invalid

This technique is applied to **income, expense and budget amount fields**.

4.2 Boundary Value Analysis

Boundary Value Analysis tests values at and around the permitted limit.

For example, if the minimum valid financial amount is ₹1:

Boundary Condition	Value	Expected Result
Below minimum	₹0	Reject
Minimum	₹1	Accept
Just above minimum	₹2	Accept

This technique helps identify errors that occur at the limits of input values.

4.3 Decision Table Testing

Decision Table Testing is useful for determining the result based on multiple conditions.

Budget Decision Table

Valid Budget	Expense Within Limit	Expected Result
Yes	Yes	Accept transaction
Yes	No	Display overspending warning
No	Yes	Reject budget
No	No	Reject and display error

This ensures that different combinations of budget and spending conditions are tested.

4.4 State Transition Testing

State Transition Testing verifies how a transaction moves between different states.

Valid Transaction

Transaction Entered



Validation



Validated



Saved



Categorized



Included in Financial Analysis

Invalid Transaction

Transaction Entered



Validation Failed



Error Message



Transaction Not Saved

This verifies that invalid financial data does not enter the system as a valid transaction.

5. REQUIREMENT AND FUNCTIONAL COVERAGE

The test cases are mapped to the major requirements of the system.

Requirement	Covered By Coverage
User Registration	TC01–TC03 ✓
User Login	TC04–TC05 ✓
Income Management	TC06–TC09 ✓
Expense Management	TC10–TC13 ✓
Budget Management	TC14–TC17 ✓
Financial Calculations	TC18–TC21 ✓
Transaction Management	TC22–TC24 ✓
AI Recommendation	TC25–TC26 ✓
Dashboard	TC27 ✓
Spending Analysis	TC28 ✓
Security	TC29–TC30 ✓
Exception Handling	TC31 ✓

5.2 Functional Coverage Summary

The test cases provide coverage for the major functions of the **AI-Based Personal Finance and Budget Planning System**:

1. **Authentication** – Registration and login are tested with valid, invalid and missing data.
2. **Income Management** – Income entry and input validation are tested.
3. **Expense Management** – Expense entry, validation and categorization are tested.
4. **Budget Planning** – Budget creation, boundary values and overspending are tested.
5. **Financial Calculation** – Income, expenses, balance and budget utilization are verified.
6. **Transaction Management** – Viewing, updating and deleting transactions are tested.
7. **AI Module** – Recommendations and insufficient-data conditions are tested.
8. **Dashboard** – Financial summary and spending analysis are verified.
9. **Security** – Unauthorized access and logout behaviour are tested.
10. **Exception Handling** – Backend failure and invalid conditions are tested.

6. CONCLUSION

The test case design for the **AI-Based Personal Finance and Budget Planning System (Banking & Finance)** covers the important functional and non-functional requirements of the system.

The test cases include **normal, boundary, invalid, and exceptional conditions**. The required fields such as **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status** are included for each test case.

Equivalence Partitioning is used to test valid and invalid financial inputs. **Boundary Value Analysis** is used for amount limits. **Decision Table Testing** is applied to budget conditions, while **State Transition Testing** verifies the movement of transactions through different processing states.

The requirement mapping demonstrates that the major functions—including authentication, income and expense management, budgeting, calculations, transaction management, AI recommendations, dashboard analysis, security, and error handling—are covered.

Therefore, the designed test cases provide **adequate requirement and functional coverage** for the AI-Based Personal Finance and Budget Planning System and help ensure that the system is accurate, secure, reliable, and suitable for personal financial management

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25